Computational Physics
Project I: Atomic Structure

Per Jönsson and Jörgen Ekman
Faculty of Technology and Society, Malmö University

Malmö 2022

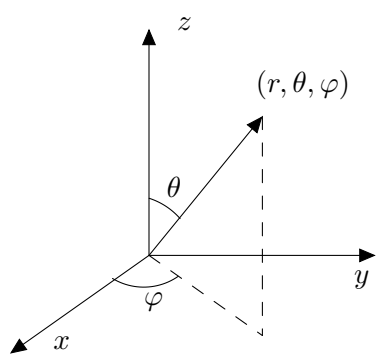
1 Introduction

We begin by looking at the Schrödinger equation for a single electron in a central potential $V(r)$. By expressing the equation in spherical coordinates, we obtain an equation governing the radial wave function. We proceed to discuss general numerical methods to obtain solutions to this equation.

The single-electron Schrödinger equation is used to describe the electronic states of an atom in the *central-field approximation*, an approximation for a many-electron system in which each electron is assumed to move independently in a central potential that accounts for the nuclear field and the field of the remaining electrons. There are various methods for determining approximate potentials. We explore some simple *parametric potentials*, but will later look at the Hartree-Fock theory. In the following section we assume that an appropriate central potential has been given and we concentrate on solving the resulting single-electron Schrödinger equation.

2 Spherical coordinates

We start by reviewing the spherical coordinates. In the figure below we have introduced the spherical coordinates (r, θ, φ).



The definition is as follows

$$\begin{cases} r = \sqrt{x^2 + y^2 + z^2} \\ \theta = \arctan(z/r) \\ \varphi = \arccos(z/r) \end{cases} \tag{1}$$

with $0 < r$, $0 \leq \theta < \pi$, $0 \leq \varphi < 2\pi$.

In radial coordinates the integration over $\mathbf{R}^3$ is

$$\iiint f(r, \theta, \varphi) r^2 \sin(\theta)\, dr d\theta d\varphi. \tag{2}$$

3 Radial Schrödinger Equation

The Schrödinger equation for a single electron in a general central potential $V(r)$ can be written

$$H\phi(\mathbf{r},\sigma) = E\phi(\mathbf{r},\sigma), \tag{3}$$

where

$$H = -\frac{\hbar^2}{2m}\nabla^2 + V(r). \tag{4}$$

The solution $\phi(\mathbf{r},\sigma)$ is referred to as a *spin-orbital*. When expressed in spherical coordinates (r,θ,φ) the equation becomes (for a detailed derivation see for example Cowan, The Theory of Atomic Structure and Spectra, pages 39 – 41)

$$\frac{1}{r^2}\frac{\partial}{\partial r}\left(r^2\frac{\partial\phi}{\partial r}\right) + \frac{1}{r^2\sin\theta}\frac{\partial}{\partial\theta}\left(\sin\theta\frac{\partial\phi}{\partial\theta}\right) + \frac{1}{r^2\sin^2\theta}\frac{\partial^2\phi}{\partial\varphi^2} + \frac{2m}{\hbar^2}\left(E - V(r)\right)\phi = 0. \tag{5}$$

We seek a solution of the form $\phi(\mathbf{r},\sigma) = \frac{1}{r}P(r)Y(\theta,\varphi)\chi_{m_s}(\sigma)$.[1] Substituting this ansatz into equation (3), we obtain the following pair of equations for the functions $P(r)$ and $Y(\theta,\varphi)$ (see appendix for a full derivation)

$$\frac{1}{\sin\theta}\frac{\partial}{\partial\theta}\left(\sin\theta\frac{\partial Y(\theta,\varphi)}{\partial\theta}\right) + \frac{1}{\sin^2\theta}\frac{\partial^2 Y(\theta,\varphi)}{\partial\varphi^2} + \lambda Y(\theta,\varphi) = 0 \tag{6}$$

$$\frac{d^2P(r)}{dr^2} + \frac{2m}{\hbar^2}\left(E - V(r) - \frac{\lambda\hbar^2}{2mr^2}\right)P(r) = 0, \tag{7}$$

where λ is a separation constant.

It can be shown that equation (5) has finite and single valued solutions only for $\lambda = \ell(\ell+1)$, $\ell = 0,1,2,\ldots$. The solutions are given by the spherical harmonics $Y_{\ell m}(\theta,\varphi)$, $\ell = 0,1,2,\ldots$ where, for each ℓ, the m quantum numbers are given by $m = -\ell, -\ell+1, \ldots, \ell$. The first few spherical harmonics are given below.

$$\begin{aligned}
Y_{00}(\theta,\varphi) &= \tfrac{1}{\sqrt{4\pi}} \\
Y_{1\pm1}(\theta,\varphi) &= \mp\sqrt{\tfrac{3}{8\pi}}\sin\theta e^{\pm i\varphi} \\
Y_{10}(\theta,\varphi) &= \sqrt{\tfrac{3}{4\pi}}\cos\theta \\
Y_{2\pm2}(\theta,\varphi) &= \sqrt{\tfrac{15}{32\pi}}\sin^2\theta e^{\pm i2\varphi} \\
Y_{2\pm1}(\theta,\varphi) &= \mp\sqrt{\tfrac{15}{8\pi}}\sin\theta\cos\theta e^{\pm i\varphi} \\
Y_{20}(\theta,\varphi) &= \sqrt{\tfrac{5}{16\pi}}(2\cos^2\theta - \sin^2\theta)
\end{aligned}$$

[1] $\chi_{m_s}(\sigma)$ is the spin function. It is of importance when constructing wave functions for many electron systems and when looking at relativistic corrections.

The spherical harmonics are orthonormal

$$\int_{\varphi=0}^{2\pi}\int_{\theta=0}^{\pi} Y^*_{\ell m}(\theta,\varphi)Y_{\ell' m'}(\theta,\varphi)\sin\theta\, d\theta d\varphi = \delta_{\ell\ell'}\delta_{mm'} \tag{8}$$

and make up a complete set with respect to the angular coordinates.

With the separation constant $\lambda = \ell(\ell+1)$ the radial Schrödinger equation becomes

$$\frac{d^2P(r)}{dr^2} + \frac{2m}{\hbar^2}\left(E - V(r) - \frac{\ell(\ell+1)\hbar^2}{2mr^2}\right)P(r) = 0. \tag{9}$$

It is convenient to introduce atomic units. Atomic units are defined by requiring that the electron's mass m, the electron's charge $|e|/\sqrt{4\pi\epsilon_0}$, and Planck's constant $\hbar$, all have the value 1. The atomic unit of length is the Bohr radius, $a_0 = 4\pi\epsilon_0\hbar^2/(me^2) = 0.529177$ Å and the atomic unit of energy is $me^4/(4\pi\epsilon_0\hbar)^2 = 27.2114$ eV. In atomic units the radial equation is

$$\frac{d^2P(r)}{dr^2} + 2\left(E - V(r) - \frac{\ell(\ell+1)}{2r^2}\right)P(r) = 0. \tag{10}$$

We study bound state solutions to the radial Schrödinger equation (7) that satisfy the normalization condition

$$\int_0^\infty P^2(r)\, dr = 1. \tag{11}$$

Such bound state solutions exist only for certain discrete energy values $E < 0$, the *energy eigenvalues*. For general potentials $V(r)$ fulfilling the condition

$$\lim_{r\to 0} rV(r) = c, \tag{12}$$

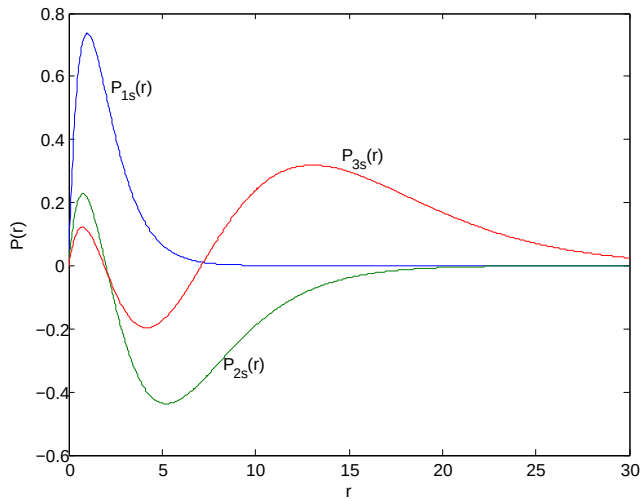
where c is a negative constant, and approaching zero at least as rapidly as $-1/r$ as $r \to \infty$, it can be shown that the different radial solutions belonging to these discrete energy eigenvalues can be distinguished by the value ℓ to which they correspond and by the number of zeros of the solutions away from the origin. A zero of the solution is often referred to as a *node*. For a given ℓ the solution belonging to the lowest energy has zero nodes, the solution belonging to the second lowest energy has one node etc. It is customary to denote the orbital angular momentum with the spectroscopic notation

$$\begin{aligned}\ell =& 0\ 1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ldots\\ & s\ p\ d\ f\ g\ h\ i\ k\ l\ldots\end{aligned} \tag{13}$$

and to use a principal quantum number n defined by

$$n = \ell + \nu + 1, \tag{14}$$

where ν is the number of nodes of the radial function not counting the zero at the origin. As an example $P_{2s}(r)$ corresponds to an orbital angular quantum



Figur 1: *Solutions with the same ℓ are distinguished by the number of nodes away from $r = 0$. $P_{1s}(r)$ has zero nodes, $P_{2s}(r)$ has one, $P_{3s}(r)$ has two etc.*

number $\ell = 0$ that has one node away from the origin, see figure 1. Since the solutions can be labeled by n and l then, for the consistency, we also label the discrete energies by these quantum numbers, i.e. $E_{n\ell}$.

We end this section by look at the limiting solutions of (7) at $r = 0$ and at the infinity. Near $r = 0$ we can neglect E and $V(r)$ compared with $\ell(\ell+1)/r^2$ and the solutions take on one of the following limiting forms

$$P_{nl}(r) = \begin{cases} r^{\ell+1} & \text{regular at the origin} \\ r^{-\ell} & \text{irregular at the origin.} \end{cases} \tag{15}$$

Since we seek normalizable solutions, we must require that our solutions be of the first type, regular at the origin. Since the potential vanishes as $r \to \infty$, it follows that

$$P_{nl}(r) = \begin{cases} e^{-\sqrt{2|E|}\,r} & \text{regular at infinity} \\ e^{\sqrt{2|E|}\,r} & \text{irregular at infinity} \end{cases} . \tag{16}$$

Again, the normalizability constraint forces us to seek solutions of the first type, regular at infinity.

4 Summary One-electron Systems

The bound state solutions to the Schrödinger equation for a single electron in a general central potential $V(r)$ are given by spin-orbitals. The spin-orbitals are characterized by the four quantum numbers $\alpha = n\ell m_\ell m_s$. Introducing the notation $\phi(\alpha, \mathbf{q})$ for the spin-orbital, where $\mathbf{q} = (\mathbf{r}, \sigma)$ we have

$$\phi(\alpha, \mathbf{q}) = \frac{1}{r} P_{n\ell}(r) Y_{\ell m_\ell}(\theta, \varphi) \chi_{m_s}(\sigma). \tag{17}$$

The radial function $P_{n\ell}(r)$ is the solution to the radial equation

$$\frac{d^2 P_{n\ell}(r)}{dr^2} + 2\left(E - V(r) - \frac{\ell(\ell+1)}{2r^2}\right) P_{n\ell}(r) = 0. \tag{18}$$

that has ν nodes away from zero, where

$$n = \ell + \nu + 1. \tag{19}$$

The corresponding energies depends on n and ℓ and are labeled E_{nl}.

In the general case the radial equation needs to be solved numerically. For the spin-orbital to be normalized $P_{n\ell}(r)$ should satisfy

$$\int_0^\infty P_{n\ell}(r)^2 \, dr = 1. \tag{20}$$

Even for normalized spin-orbitals there is a trivial arbitrariness between $P_{n\ell}(r)$ and $-P_{n\ell}(r)$. To resolve this we use the convention that radial functions should be positive near the origin.

5 Analytical Solution to the Radial Equation

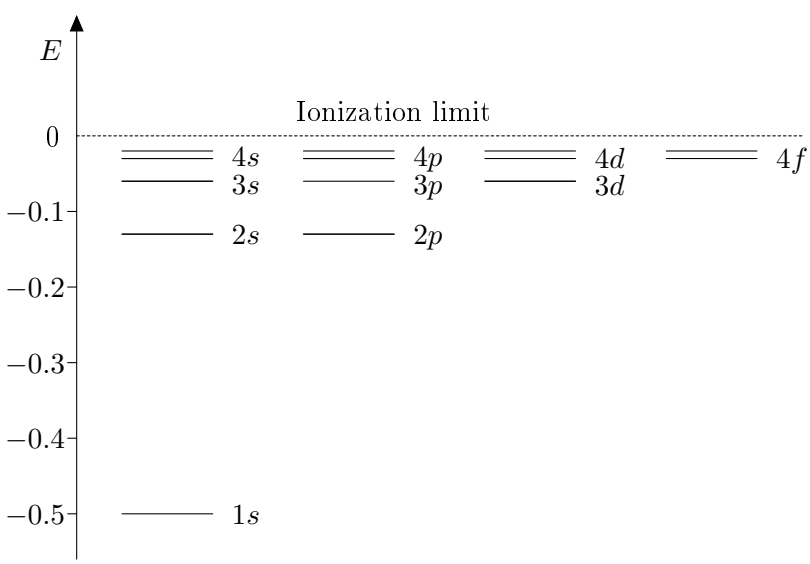
In a Coulomb field $V(r) = -Z/r$ the radial Schrödinger equation can be solved analytically. The energies depend only on the principal quantum number n, and not on the angular quantum number ℓ, and are given by

$$E_n = -\frac{Z^2}{2n^2}, \quad n = 1, 2, 3, \ldots \tag{21}$$

The corresponding radial functions are products of the regular solutions at the origin and the infinity and some polynomial in r. All details can be found in the book of Cowan pages 70 – 71. Below we give the expressions for some of the radial functions belonging to the lowest energies.

$$
\begin{aligned}
P_{1s}(r) &= 2Z^{3/2}re^{-Zr} \\
P_{2s}(r) &= \frac{1}{\sqrt{2}}Z^{3/2}re^{-Zr/2}\left(1-\frac{1}{2}Zr\right) \\
P_{2p}(r) &= \frac{1}{2\sqrt{6}}Z^{5/2}r^2e^{-Zr/2} \\
P_{3s}(r) &= \frac{2}{3\sqrt{3}}Z^{3/2}re^{-Zr/3}\left(1-\frac{2}{3}Zr+\frac{2}{27}Z^2r^2\right) \\
P_{3p}(r) &= \frac{8}{27\sqrt{6}}Z^{5/2}r^2e^{-Zr/3}\left(1-\frac{1}{6}Zr\right) \\
P_{3d}(r) &= \frac{4}{81\sqrt{30}}Z^{7/2}r^3e^{-Zr/3}
\end{aligned}
$$

For hydrogen the energies are degenerate meaning that there are several states having the same energy. The states are often arranged in a diagram similar to the on below with the energy on one axis and the quantum states arrange according to the quantum numbers on the other axis. Energies are relative to the ionization limit at which the electron is at rest at infinite distance from the nucleus.



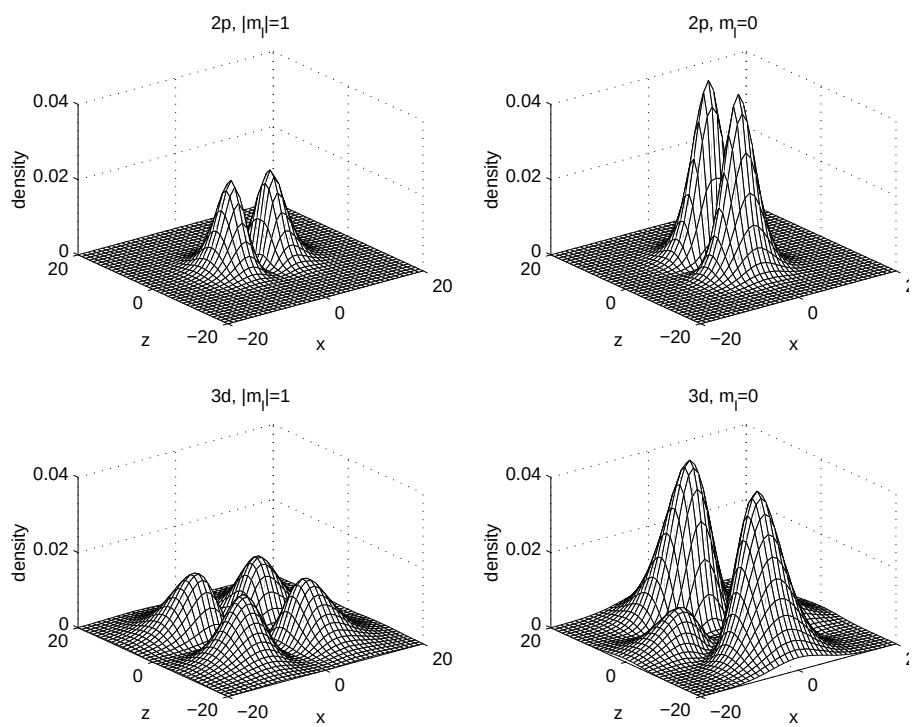
Figur 2: *Energy diagram for hydrogen. States are arranged according to ℓ symmetry and energies are given relative to the ionization limit.*

6 Graphical Representation of Charge Densities

According to the statistical interpretation of quantum mechanics the electron density, which equals the spin independent probability density, is given by

$$|R_{n\ell}(r)Y_{\ell m_\ell}(\theta,\varphi)|^2. \tag{22}$$

Using the formulas for the spherical harmonics it is seen that this quantity is independent of the azimuthal angle φ, and that there is a rotational symmetry around the z-axis. In figure 3 we display the charge density in the xz-plane for a few electron states in hydrogen.



Figur 3: Charge densities in the xz-plane for the $2p$, $m_l = \pm 1, 0$ and $3d$, $m_l = \pm 1, 0$ states in hydrogen. There is a rotational symmetry around the z-axis.

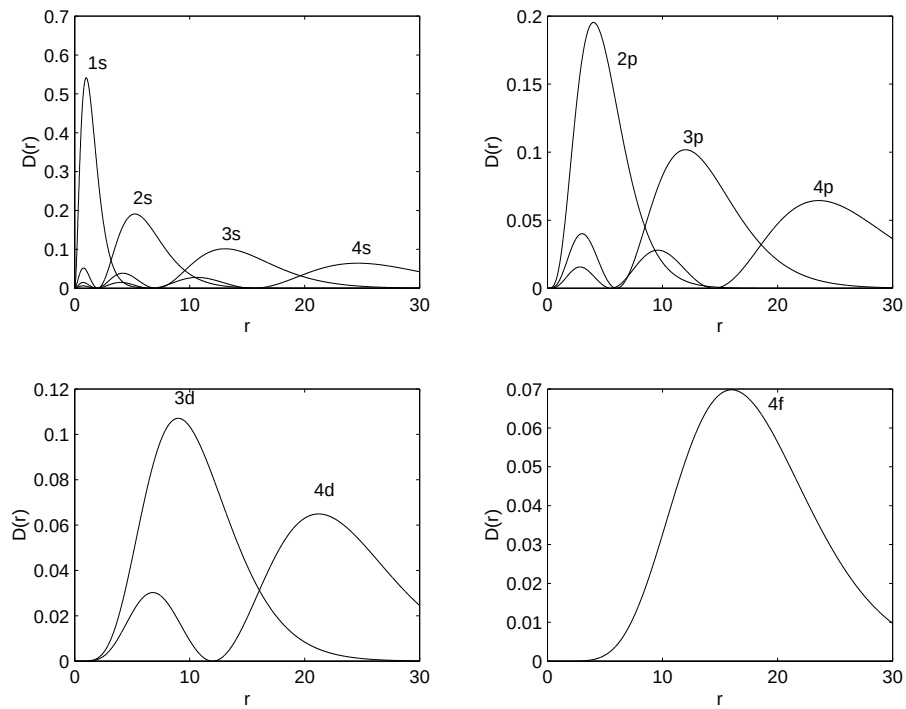
Another important measure of the charge density is the *radial density function* $D(r)$, which gives the probability per unit length of finding the electron at the distance r from the nucleus. The radial density function is obtained by integrating the charge density over the angular co-ordinates

$$D(r) = R_{n\ell}^2(r)r^2 \iint |Y_{lm_l}(\theta,\varphi)|^2 \sin\theta \, d\theta d\varphi. \tag{23}$$

Since the angular integral equals one this expression simplifies to

$$D(r) = R_{nl}^2(r)r^2 = P_{nl}^2(r). \tag{24}$$

Some of the radial densities for hydrogen are depicted in figure 2.



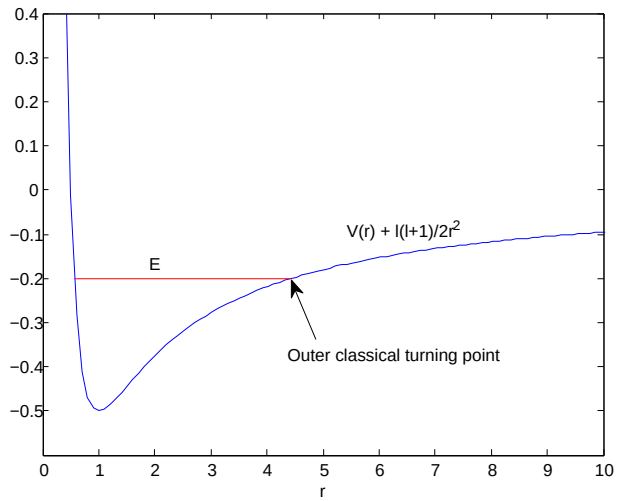
Figur 4: *Radial charge densities for hydrogen. Note that only s electrons have finite charge densities at the origin and that other orbitals are kept away from the nucleus by the* $\frac{\ell(\ell+1)}{2r^2}$ *term in the radial part of the SE, compare also exercise 9.*

Note that for any given n, the probability of finding the electron close to the nucleus is highest for states having low values of ℓ. This is consistent with the semi-classical picture of the electron moving in an elliptic orbit that is eccentric for low ℓ and more circular for high ℓ (see Cowan pages 80 – 81).

7 Numerical Solution to the Radial Equation

Our problem is to determine the energy eigenvalues and the associated radial functions, or eigenfunctions, $P_{n\ell}(r)$ in the case of a general potential $V(r)$. We use Cowell-Numerov's method (to be described) to find solutions to the radial equation on a finite grid covering the region $r = 0$ to a practical infinity, r_∞, a point determined by requiring that $P_{n\ell}(r)$ be negligible for $r > r_\infty$. The practical infinity depends on the eigenvalue E and is typically chosen so that $\sqrt{2|E|} \cdot r_\infty \approx 40$.

Near the origin, there are two solutions to the radial Schrödinger equation, the desired solution which behaves as $r^{\ell+1}$, and an irregular solution, referred to as the complementary solution, which diverges as $r^{-\ell}$ as $r \to 0$. Numerical errors near $r = 0$ always introduce small admixtures of the complementary solution into the solution being sought. Integrating outward from the origin keeps such



Figur 5: *En electron with energy E in a potential $V(r) + \frac{\ell(\ell+1)}{2r^2}$ is classically allowed to move only inside a certain region.*

errors under control, since the complementary solution decreases in magnitude as r increases. Continuing the integration toward the practical infinity we will have admixtures of the irregular solution $e^{\sqrt{2|E|}r}$. Now, this solution will blow up and totally destroy our final result, i.e. the integration process is numerically unstable. Starting from the regular solution at the practical infinity integrating inwards we will close to $r = 0$ have admixtures of the irregular solution $r^{-\ell}$ which in a similar way will blow up and destroy the solution.

To avoid the above problems, the general procedure to solve the radial equation is to integrate outward from the origin, using Cowell-Numerov's method, starting with solutions that are regular at the origin. The integration is continued to the outer classical turning point, the point beyond which classical motion in the potential

$$V(r) + \frac{\ell(\ell+1)}{2r^2} \tag{25}$$

is impossible, see figure 4. In the region beyond the classical turning point, the equation is integrated inward, using Cowell-Numerov's method, starting from $r = r_\infty$ with an approximate solution obtained from the solution regular at the infinity. The solutions from the inward and outward integrations are matched at the classical outer turning point. The energy is then adjusted until the derivative of $P_{n\ell}(r)$ is continuous at the matching point. The resulting function $P_{n\ell}(r)$ is an eigenfunction and the corresponding energy E is its eigenvalue. The radial function is then normalized to fulfill $\int_0^\infty P_{n\ell}(r)^2\, dr = 1$.

8 The Cowell-Numerov's Method

We shall consider the Cowell-Numerov's method for solving equations of the form

$$y'' = f(x, y). \tag{26}$$

Introducing the notation $x_k = x_0 + kh$ for the uniformly space grid where h is the step and $y_k = \{$ the value of y obtained from the method at grid point $x_k\}$ the solution is given by the formula

$$y_{k+1} = 2y_k - y_{k-1} + \frac{h^2}{12}(f_{k+1} + 10f_k + f_{k-1}) \tag{27}$$

with a local truncation error $\mathcal{O}(h^6)$. The formula above is implicit but in the special case when $f(x, y) = g(x)y$, as for the radial equation, we get the following explicit formula for the outward integration

$$y_{k+1} = \frac{\beta_k y_k - \alpha_{k-1} y_{k-1}}{\alpha_{k+1}} \tag{28}$$

where $\alpha_k = 1 - (h^2/12)\, g_k$ and $\beta_k = 2 + (5h^2/6)\, g_k$. Rearranging we have

$$y_{k-1} = \frac{\beta_k y_k - \alpha_{k+1} y_{k+1}}{\alpha_{k-1}} \tag{29}$$

for the inward integration. To start the method we need two values. These can be inferred from, respectively, the regular solution at the origin and at the infinity with additional corrective terms. For simplicity we will drop higher order terms.

9 General Numerical Procedure

For a given potential $V(r)$ and a given ℓ we obtain a solution $P_{n\ell}(r)$ to the radial equation with a specified number of nodes away from $r = 0$ by following the steps below:

1. Start with the lowest possible energy eigenvalue $E = -Z^2/(2n^2)$. Increase energy until the classical turning point is in the interior of the grid, see figure 4. This energy will be the starting point for further calculations.

2. Use the Cowell-Numerov's method with starting values from a low order series expansion $P_{n\ell}(r) \propto r^{\ell+1}\left(1 - \frac{Zr}{\ell+1}\right)$ of the regular solution to obtain values of the radial function at a number of grid points from $r = 0$ out to the classical turning point r_c (outward solution).

3. Use the Cowell-Numerov's method with starting values from the regular solution to obtain values of the radial function at a number of grid points from the practical infinity r_∞ in to the classical turning point r_c (inward solution).

4. Multiply the wave function obtained in step 3 by a scale factor chosen to make the wave function for $r < r_c$ from step 2, and that for $r > r_c$ from step 3, continuous at $r = r_c$.

Since we have guessed the energy E in step 1 the solution may have too many many nodes (initial energy too high) or too few nodes (initial energy too low). In the first case we reduce the energy by multiplying with a factor of 1.1 and redo steps 2 – 4. In the second case we increase the energy by multiplying with a factor 0.9 and redo steps 2 – 4. This process is continued until we have the desired number of nodes. In the routine, we keep track of the least upper bound on the energy E_u (too many nodes) and the greatest lower bound E_l (too few nodes) as the iteration proceeds. If increasing the energy at a particular step of the iteration would lead to $E > E_u$, then we simply replace E by $(E + E_u)/2$, rather than following the above rules. Similarly, if decreasing E would lead to $E < E_l$, then we replace E by $(E + E_l)/2$.

Even if the solution after the steps above has the correct number of nodes the derivative may still be discontinuous at the matching point. To achieve a wave function with a continuous derivative, we make further corrections to E using a perturbative approach. If we let $P_1(r)$ and $P_1'(r)$ represent the radial wave function and its derivative at E_1, respectively, and let $P_2(r)$ and $P_2'(r)$ represent the same two quantities at E_2, then it follows from the radial Schrödinger equation that

$$2(E_1 - E_2)P_1P_2 = \frac{d}{dr}\left(P_2'P_1 - P_2P_1'\right). \tag{30}$$

From this equation, we find that

$$2(E_1 - E_2)\int_{r_c}^{\infty} P_1P_2\,dr = \left[P_2'P_1 - P_2P_1'\right]_{r_c}^{\infty} = -\left(P_2'P_1 - P_2P_1'\right)^{+} \tag{31}$$

$$2(E_1 - E_2)\int_{0}^{r_c} P_1P_2\,dr = \left[P_2'P_1 - P_2P_1'\right]_{0}^{r_c} = \left(P_2'P_1 - P_2P_1'\right)^{-} \tag{32}$$

where the superscripts $\pm$ indicate that the quantities in parentheses are to be evaluated just above or just below r_c. These equations are combined to give

$$E_1 - E_2 = \frac{(P'^{+}_1 - P'^{-}_1)P_2(r_c) + (P'^{-}_2 - P'^{+}_2)P_1(r_c)}{2\int_0^{\infty} P_1P_2\,dr} \tag{33}$$

Suppose that the derivative P_1' is discontinuous at r_c. If we demand that P_2' be continuous at r_c, then the term $P'^{-}_2 - P'^{+}_2$ in the numerator of (31) vanishes. Approximating P_2 by P_1 in this equation, we obtain

$$E_2 \approx E_1 + \frac{(P'^{-}_1 - P'^{+}_1)P_1(r_c)}{2\int_0^{\infty} P_1^2\,dr}. \tag{34}$$

as an approximation for the eigenenergy. We use this approximation iteratively until the discontinuity in $P'(r)$ at $r = r_c$ is reduced to an insignificant level.

As a last step we normalize the solution and make sure that it has a positive slope at the origin.

10 Python Implementation

Below is a Python implementation of the general numerical procedure to solve the radial Schrödinger equation in a Coulomb potential $V(r) = -Z/r$. We can target specific solutions by giving n and ℓ. The user is supposed to supply the nuclear charge Z, the number of grid points m as well as some starting energy E. The routine fails if the initial $E > 0$ or if E less than the smallest value of the potential $V(r) + \frac{\ell(\ell+1)}{2r^2}$, i.e. if no classical turning point can be found (see figure 4). In implementing the solution we have prioritized simplicity rather than accuracy. To increase the numerical accuracy the starting values for the outward and inward integration should be improved by adding higher order correction terms. Most important, we should have used a logarithmic grid rather than a uniform one. Paying attention to the above things an accuracy of parts in 10^{12} can be achieved with 500 grid points. For the present implementation we use as many as 10000 grid points to get a reasonable accuracy for $\ell = 0$.

```
# Program radial.py
# Program for solving the radial Schrödinger equation for a Coulomb potential V(r)=-Z/r and a
# given quantum number l. A specific solution is targeted by giving the principal quantum number n.
#                                                                    Per Jönsson, September 2011
# Matlab to python conversion by Asimina Papoulia and Wenxian Li, August 2020

import math
import numpy as np
import matplotlib.pyplot as plt
import sys
import datetime

def radiallog(l,n,Z,plot=True):

    starttime=datetime.datetime.now().timestamp()

    if Z > 0 and l < n:
        E = - Z**2/2                        #starting energy taken as the lowest possible
        nodes = n - l - 1                   #targeted number of nodes
    else:
        sys.stderr.write('Invalid input\n')
        sys.exit(-1)

    grid_points = 10000                     #number of grid points
    nodes_count = -1                        #initial values of nodes
    dE = math.inf                           #initial values of difference of derivat at r_c
    E_upper = -2.0e-52                      #initial values of upper bound of energy during iter
    E_lower = -math.inf                     #initial values of lower bound of energy during iter
    num_iter = 0                            #initial values of interation number

    while nodes != nodes_count or abs(dE) > 1.0e-12:
        #Make sure that the energy is such that the turning point is in the interior of the grid
        while 1:
            r_inf = 40/math.sqrt(2*abs(E))              #practical infinity
            r = np.linspace(0, r_inf, grid_points)      #generate grid
            U = np.linspace(0, 0, grid_points)          #initialize U
            U[0] = 0                                    #gets a  0  value at r = 0
            U[1:] = -Z/r[1:] + l*(l+1)/(2*r[1:]**2)
            #Determine the outer classical turning point. Start from the practical infinity
```

```
    # and step inwards until U(i) < E
    i_c = grid_points-1
    for i in range(grid_points-1, -1, -1):
        if U[i] < E:
            i_c = i
            break
    #If turning point not in the interior increase the energy and try again
    if i_c == grid_points-1:
        E = 0.9*E
    else:
        break

h = r_inf/(grid_points-1)                          #step size
g = -2*(E-U)                                       #g function for Numerov's method
alpha = 1-(h**2/12)*g                              #alpha and
beta = 2+(5*h**2/6)*g                              #beta for Numerov's method

#Perform the outward integration
P_out = np.zeros(grid_points)                      #initialize P_out
P_out[0] = r[0]**(l+1)*(1-Z*r[0]/(l+1))            #starting values from regular
P_out[1] = r[1]**(l+1)*(1-Z*r[1]/(l+1))            #solution at r = 0
for i in range(1,i_c+1):
    P_out[i+1] = (beta[i]*P_out[i] - alpha[i-1]*P_out[i-1])/alpha[i+1]

#Perform the inward integration
P_in = np.zeros(grid_points)                                        #initialize P_in
P_in[grid_points-1] = np.exp(-np.sqrt(2*abs(E)*r[grid_points-1]))   #starting values from
P_in[grid_points-2] = np.exp(-np.sqrt(2*abs(E)*r[grid_points-2]))   #regular solution at r_inf
for i in range(grid_points-2,i_c-1,-1):
    P_in[i-1] = (beta[i]*P_in[i] - alpha[i+1]*P_in[i+1])/alpha[i-1]

#Scale and merge solutions
P_out = P_out/P_out[i_c]
P_in = P_in/P_in[i_c]
P = np.append(P_out[0:i_c], P_in[i_c:])

#Count the number of nodes. At a node position, P(i+1)P(i) is negative
nodes_count = 0
for i in range(1,grid_points-1):
    if P[i+1]*P[i] < 0:
        nodes_count +=1

#Check nodes and adjust E
if nodes_count > nodes:                            #too many nodes, decrease E
    E_upper = min(E_upper,E)
    if E*1.1 < E_lower:
        E = (E_lower+E)/2
    else:
        E = E*1.1
elif nodes_count < nodes:                          #too few nodes, increase E
    E_lower = max(E_lower,E)
    if E*0.9 > E_upper:
        E = (E+E_upper)/2
    else:
        E = E*0.9

#Fine tune the energy
```

```
        if nodes_count == nodes:
            #Compute the derivatives and take difference
            DP_out = (P_out[i_c+1]-P_out[i_c-1])/(2*h)
            DP_in = (P_in[i_c+1]-P_in[i_c-1])/(2*h)
            dE = (DP_out-DP_in)/(2*sum(P**2)*h)
            #Damp if step is too large
            while E+dE > 0:
                dE = dE/2
            E = E + dE
        print("Iteration %d: E = %.16e" %(num_iter,E,))
        num_iter += 1

    #Normalize and check slope
    norm = math.sqrt(sum(P**2)*h)
    P = P/norm
    if P[1] < 0:
        P = -P

    #Compute radial expectation value
    r1 = sum(r*P**2)*h

    endtime=datetime.datetime.now().timestamp()
    print("time used: %f s" %(endtime-starttime))

    #Write energy and plot solution
    print()
    print("Number of iterations: %d" %(num_iter,))
    print("Energy eigenvalue: %.16e a.u." %(E,))
    print("Radial expectation value: %.16e a.u." %(r1,))

    orbital=["s","p","d","f","g","h","i"]

    if plot:
        plt.figure()
        plt.subplot(211)
        plt.plot(r,P)
        plt.xlabel("r (a.u.)")
        plt.title("radial function for %d%s of Z = %d" %(n,orbital[l],Z,))
        plt.ylabel("P")
        rr = np.sqrt(r)
        plt.subplot(212)
        plt.plot(rr,P)
        plt.xlabel('$\sqrt{\mathrm{r}}$ (a.u.)')
        plt.ylabel("P")
        plt.tight_layout(pad=1.0)
        plt.show()

    return (r,P,)

if __name__ == '__main__':
    #input
    l = int(input("Give the orbital quantum number l: "))
    n = int(input("Give the principal quantum number n: "))
    Z = int(input("Give the nuclear charge Z: "))

    radiallog(l,n,Z)
```

11 Transformation to Logarithmic Grid

To achieve high numerical accuracy it is essential that the gridpoints are densely spaced at the origin. Further away from the origin the distance from between the gridpoins can be larger. At the same time numerical procedures are much simplified if the grid is equally spaced. To deal with both aspects a transformation of the independent variable

$$\rho = \log(Zr) \to r = \frac{e^{\rho}}{Z} \tag{35}$$

and the dependent variable

$$\overline{P}(\rho) = \frac{P(r)}{\sqrt{r}} \tag{36}$$

has been suggested (Froese-Fischer, Brage and Jönsson, Computational Atomic Structure, p. 58). With this transformation the radial equation becomes

$$\frac{d^2\overline{P}_{n\ell}(\rho)}{d\rho^2} + \left(2r^2[E - V(r)] - \left(\ell + \frac{1}{2}\right)^2\right)\overline{P}_{n\ell}(\rho) = 0, \tag{37}$$

with the boundary conditions $\overline{P}_{n\ell}(\rho) \to 0$ as $\rho \to \pm\infty$. In order to avoid points at $-\infty$ it is customary to divide the range in r in two regions $[0, r_1]$ and $[r_1, \infty[$ and transform only the latter to the ρ variable. The value r_1 is sufficiently small that a series expansion may be used for the solution near the origin.

Test shows that appropriate values are $\rho_1 = -10$ and $h = 1/48$. This gives the grid

$$\rho_i = -10 + (i-1)/48 \qquad \text{and} \qquad r_i = \frac{e^{\rho_i}}{Z}, \qquad i = 1, 2, \ldots, M \tag{38}$$

Near the origin the series expansion

$$P_{n\ell}(r) \propto r^{\ell+1}\left(1 - \frac{Zr}{\ell+1}\right) \tag{39}$$

is used. The previous Python code can, with small modifications, be used also in this case. The accuracy is much better than for a linear grid. The transformation implies a modification of the update formula for the energy so that

```
DP_out = (P_out[i_c+1]-P_out[i_c-1])/(2*h)
DP_in = (P_in[i_c+1]-P_in[i_c-1])/(2*h)
dE = (DP_out-DP_in)*P[i_c]/(2*sum(r**2*P**2)*h)
```

Also, the expressions for normalization and radial expectation values are

```
norm = math.sqrt(sum(P**2*r**2)*h)
```

and

```
r1 = sum(r**3*P**2)*h
```

where now P is $\overline{P}$. The program based on the logaritmic grid is found below.

```
# Program radiallog.py
# Program for solving the radial Schrödinger equation for a Coulomb potential V(r)=-Z/r and a
# given quantum number l. A transformation to logaritmic grid is done to increase numerical
# accuracy. A specific solution is targeted by giving the principal quantum number n.
#                                                    Jörgen Ekman and Per Jönsson, April 2015
# Matlab to python conversion by Asimina Papoulia and Wenxian Li, September 2020

import math
import numpy as np
import matplotlib.pyplot as plt
import sys
import datetime

def radiallog(l,n,Z,plot=True):

    starttime=datetime.datetime.now().timestamp()

    if Z > 0 and l < n:
        E = - Z**2/2                        #starting energy taken as the lowest possible
        nodes = n - l - 1                   #targeted number of nodes
    else:
        sys.stderr.write('Invalid input\n')
        sys.exit(-1)

    rhomin = -10
    h = 1/48                                #step size
    nodes_count = -1                        #initial values of nodes
    dE = math.inf                           #initial values of difference of derivat at r_c
    E_upper = -2.0e-52                      #initial values of upper bound of energy during iter
    E_lower = -math.inf                     #initial values of lower bound of energy during iter
    num_iter = 0

    while nodes != nodes_count or abs(dE) > 1.0e-10:
        #Make sure that the energy is such that the turning point is in the interior of the grid
        while 1:
            r_inf = 40/math.sqrt(2*abs(E))                              #practical infinity
            rhoi = math.log(Z*r_inf);                                   # practical infinity in rho
            grid_points = round((-rhomin + rhoi)/h);                    # number of grid points
            rho = rhomin + np.linspace(0,grid_points-1,grid_points)*h;  # generate grid
            r = np.exp(rho)/Z;
            # Define the effective potential
            U = np.linspace(0, 0, grid_points)            #initialize U
            U[0] = 0                                      #gets a  0  value at r = 0
            U[1:] = -Z/r[1:] + l*(l+1)/(2*r[1:]**2)

            #Determine the outer classical turning point. Start from the practical infinity
            # and step inwards until U(i) < E
            i_c = grid_points-1
            for i in range(grid_points-1, -1, -1):
                if U[i] < E:
                    i_c = i
                    break
            #If turning point not in the interior increase the energy and try again
            if i_c == grid_points-1:
                E = 0.9*E
            else:
```

```
        break

g = -2*r**2*(E + Z/r) + (l+1/2)**2;                #g function for Numerow
alpha = 1-(h**2/12)*g                              #alpha and
beta = 2+(5*h**2/6)*g                              #beta for Numerov's method

#Perform the outward integration
P_out = np.zeros(grid_points)                                  #initialize P_out
P_out[0] = r[0]**(l+1)*(1-Z*r[0]/(l+1))/math.sqrt(r[0])     #starting values from regular
P_out[1] = r[1]**(l+1)*(1-Z*r[1]/(l+1))/math.sqrt(r[1])     #solution at r = 0
for i in range(1,i_c+1):
    P_out[i+1] = (beta[i]*P_out[i] - alpha[i-1]*P_out[i-1])/alpha[i+1]

#Perform the inward integration
#initialize P_in
P_in = np.zeros(grid_points)
#starting values from
P_in[grid_points-1] = np.exp(-np.sqrt(2*abs(E)*r[grid_points-1]))/math.sqrt(r[grid_points-1])
#regular solution at r_inf
P_in[grid_points-2] = np.exp(-np.sqrt(2*abs(E)*r[grid_points-2]))/math.sqrt(r[grid_points-2])
for i in range(grid_points-2,i_c-1,-1):
    P_in[i-1] = (beta[i]*P_in[i] - alpha[i+1]*P_in[i+1])/alpha[i-1]

#Scale and merge solutions
P_out = P_out/P_out[i_c]
P_in = P_in/P_in[i_c]
P = np.append(P_out[0:i_c], P_in[i_c:])

#Count the number of nodes. At a node position, P(i+1)P(i) is negative
nodes_count = 0
for i in range(1,grid_points-1):
    if P[i+1]*P[i] < 0:
        nodes_count +=1

#Check nodes and adjust E
if nodes_count > nodes:                            #too many nodes, decrease E
    E_upper = min(E_upper,E)
    if E*1.1 < E_lower:
        E = (E_lower+E)/2
    else:
        E = E*1.1
elif nodes_count < nodes:                          #too few nodes, increase E
    E_lower = max(E_lower,E)
    if E*0.9 > E_upper:
        E = (E+E_upper)/2
    else:
        E = E*0.9

#Fine tune the energy
if nodes_count == nodes:
    #Compute the derivatives and take difference
    DP_out = (P_out[i_c+1]-P_out[i_c-1])/(2*h)
    DP_in = (P_in[i_c+1]-P_in[i_c-1])/(2*h)
    dE = (DP_out-DP_in)*P[i_c]/(2*sum(r**2*P**2)*h)
    #Damp if step is too large
    while E+dE > 0:
        dE = dE/2
```

```
            E = E + dE
        print("Iteration %d: E = %.16e" %(num_iter,E,))
        num_iter += 1

    #Normalize and check slope
    norm = math.sqrt(sum(P**2*r**2)*h)
    P = P/norm
    if P[1] < 0:
        P = -P

    #Compute radial expectation value
    r1 = sum(r**3*P**2)*h

    endtime=datetime.datetime.now().timestamp()
    print("time used: %f s" %(endtime-starttime))

    #Write energy and plot solution
    print()
    print("Number of iterations: %d" %(num_iter,))
    print("Number of grid points: %d" %(grid_points,))
    print("Energy eigenvalue: %.16e a.u." %(E,))
    print("Radial expectation value: %.16e a.u." %(r1,))

    orbital=["s","p","d","f","g","h","i"]

    if plot:
        plt.figure()
        plt.subplot(211)
        plt.plot(r,np.sqrt(r)*P)
        plt.xlabel("r (a.u.)")
        plt.title("radial function for %d%s of Z = %d" %(n,orbital[l],Z,))
        plt.ylabel("P")
        rr = np.sqrt(r)
        plt.subplot(212)
        plt.plot(rr,np.sqrt(r)*P)
        plt.xlabel('$\sqrt{\mathrm{r}}$ (a.u.)')
        plt.ylabel("P")
        plt.tight_layout(pad=1.0)
        plt.show()

    return (r,P,)

if __name__ == '__main__':
    #input
    l = int(input("Give the orbital quantum number l: "))
    n = int(input("Give the principal quantum number n: "))
    Z = int(input("Give the nuclear charge Z: "))

    radiallog(l,n,Z)
```

12 Central-field Approximation

For N-electron systems the exact forms of the solutions to the Schrödinger equation are not known. Instead approximate solutions must be found. One possibility that provides a lot of insight into the nature of the approximate so-

lutions is to replace the non-relativistic Hamiltonian H by one for which the Schrödinger equation is solvable.

In the *central-field approximation* the Hamiltonian H is replaced with the separable Hamiltonian H_0

$$H_0 = \sum_{i=1}^{N} \left(-\frac{1}{2}\nabla_i^2 + V(r_i) \right), \tag{40}$$

where $V(r)$ is a central potential. The potential experienced by a bound atomic electron near $r = 0$ is dominated by the nuclear Coulomb potential, so we expect

$$V(r) \approx -\frac{Z}{r} \tag{41}$$

for small r. At large r, on the other hand, an electron experiences a potential that is the sum of the attractive nuclear Coulomb potential and the sum of the repulsive potentials of the remaining electrons (screening), so we expect

$$\lim_{r\to\infty} V(r) = -\frac{\zeta}{r} \tag{42}$$

with $\zeta = Z - N + 1$ for an N-electron atom with nuclear charge Z. This limiting form is referred to as the ionic potential. It is straight forward to devise a potential $V(r)$ depending on several adjustable parameters that accounts for screening and interpolates between the inner nuclear potential and outer ionic potential. We will come back to this in section 14.

Since H_0 is separable the equation

$$H_0\psi_0(\mathbf{q}_1, ..., \mathbf{q}_N) = E_0\psi_0(\mathbf{q}_1, ..., \mathbf{q}_N) \tag{43}$$

is easy to solve. The eigenvalues and eigenfunctions can be written

$$E_0 = \sum_{i=1}^{N} E_i \tag{44}$$

and

$$\psi_0(\mathbf{q}_1, ..., \mathbf{q}_N) = \prod_{i=1}^{N} \phi(\alpha_i; \mathbf{q}_i), \tag{45}$$

where the individual spin-orbitals are solutions to the one-electron equation

$$\left[-\frac{1}{2}\nabla^2 + V(r) \right] \phi(\alpha; \mathbf{q}) = E\phi(\alpha; \mathbf{q}) \tag{46}$$

As previously shown, the one-electron function, or spin-orbital, can be written

$$\phi(\alpha; \mathbf{q}) = \frac{1}{r} P_{n\ell}(r) Y_{\ell m_\ell}(\theta, \varphi) \chi_{m_s}(\sigma), \tag{47}$$

where the solutions are characterized by the quantum numbers $\alpha = n\ell m_\ell m_s$.

The Hamiltonian H_0 is invariant with respect to permutations of the electron coordinates and so any permutation of the co-ordinates in the product function also leads to an eigenfunction. By combining the permuted product functions we can form an antisymmetric function

$$\Phi(\mathbf{q}_1, ..., \mathbf{q}_N) = \mathcal{A} \prod_{i=1}^{N} \phi(\alpha_i; \mathbf{q}_i). \tag{48}$$

This function can also be represented as a so called *Slater determinant*

$$\Phi(\mathbf{q}_1, ..., \mathbf{q}_N) = \frac{1}{\sqrt{N!}} \begin{vmatrix} \phi(\alpha_1; \mathbf{q}_1) & \phi(\alpha_1; \mathbf{q}_2) & ... & \phi(\alpha_1; \mathbf{q}_N) \\ \phi(\alpha_2; \mathbf{q}_1) & \phi(\alpha_2; \mathbf{q}_2) & ... & \phi(\alpha_2; \mathbf{q}_N) \\ \vdots & \vdots & & \vdots \\ \phi(\alpha_N; \mathbf{q}_1) & \phi(\alpha_N; \mathbf{q}_2) & ... & \phi(\alpha_N; \mathbf{q}_N) \end{vmatrix}. \tag{49}$$

In this representation it is seen that the total wave function $\Phi(\mathbf{q}_1, ..., \mathbf{q}_N)$ vanishes identically if two electrons have the same value of the four quantum numbers $\alpha = n\ell m_\ell m_s$. Thus, for allowed states of the atom no two electrons can have the same value of the four quantum numbers. This is the *exclusion principle* in the form originally discovered by Pauli (1925). Note also that the determinant vanishes whenever $\mathbf{q}_i = \mathbf{q}_j$; that is, whenever two electrons with the same spin have the same space co-ordinates.

To determine the parity of the Slater determinant we use that each of the spin-orbitals building the Slater determinant has parity $(-1)^\ell$. The Slater determinant itself must therefore have the well defined parity

$$\pi = (-1)^{\ell_1}(-1)^{\ell_2}...(-1)^{\ell_N} = (-1)^{\sum_{i=1}^{N} \ell_i} \tag{50}$$

which is even or odd dependent on whether the sum of the orbital angular momentum quantum numbers is even or odd.

13 Electron Configuration

As seen above, an eigenfunction of the central-field Hamiltonian H_0 can be written as a Slater determinant. The corresponding energy E_0 is then given by the sum of the energies of the spin-orbitals appearing in the determinant

$$E_0 = \sum_{i=1}^{N} E_i. \tag{51}$$

Spin-orbitals with the same values of the n and ℓ quantum numbers are said to belong to the same *subshell* (or shell, for short) and are called equivalent spin-orbitals. Correspondingly, we may loosely speak of electrons belonging to the same subshell and refer to them as *equivalent electrons*. Since the energy of a spin-orbital depends only on the n and ℓ quantum numbers, the energy E_0 is entirely determined by the *electron configuration*, that is, the distribution of

spin-orbitals with respect to the subshells. A general electron configuration is given by

$$(n_1\ell_1)^{w_1}(n_2\ell_2)^{w_2}...(n_m\ell_m)^{w_m}, \qquad N = \sum_{a=1}^{m} w_a, \tag{52}$$

where $w_1, w_2...$ etc. are the occupation numbers of the spin-orbitals in the different subshells. The corresponding energy can then be written

$$E_0 = \sum_{a=1}^{m} w_a E_{n_a\ell_a}, \tag{53}$$

where $E_{n\ell}$ denotes the energy of the spin-orbitals in an $n\ell$ subshell. Often the electron configurations are denoted by means of the spectroscopic symbols given in previous sections. For instance $1s^2 2s^2 2p^2$ means that there are two electrons in each of the $1s$, $2s$ and $2p$ subshells.

According to the Pauli exclusion principle there can be only one electron in each spin-orbital, and thus there can be at most $2(2\ell+1)$ electrons in a subshell $n\ell$. A subshell which is fully occupied is said to be *closed* in contrast to a partially occupied shell which is said to be *open*. The energy of a configuration is, according to (41), given by the occupation numbers of each subshell. Therefore, the ground configuration for a particular atom should be obtained by successively filling the electron subshells with the lowest energies $E_{n\ell}$ leading to a number of closed subshells and at most one open shell. This is the so called "auf Bau" principle formulated by Bohr (1922) to explain the periodic table of the elements.

The concept of a configuration has a simple interpretation. For light atoms, the experimental energy levels often appear in closely spaced groups (see figure 5). When a central-field calculation is performed using a suitable central potential $V(r)$, it is found that the average energy of each of these groups corresponds well with the energy of a certain configuration, and thus it is possible to assign configuration labels to the groups. Furthermore, if the assignment of the configuration has been done correctly, it is seen that the number of states [2] in a group is equal to the number of determinants of the corresponding configuration.

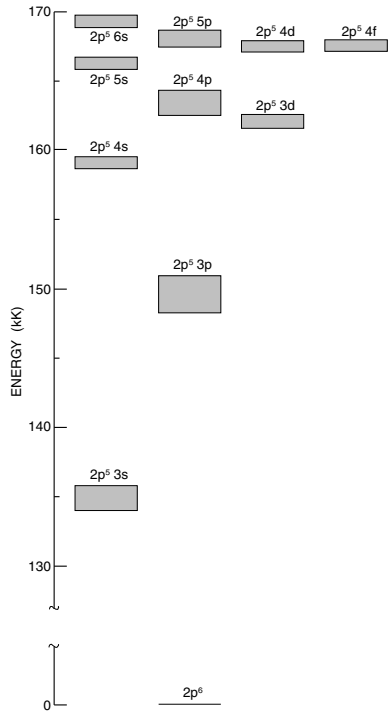
14 Parametric Potentials

Different central-potentials have been used in atomic structure calculations. Here we look at one rather simple one

$$V_a(r) = -\frac{Z}{r} + \frac{(Z-\zeta)r}{a^2+r^2} \tag{54}$$

that depends on one parameter a. The parameter is adjusted so that the computed energies in the central-field approximation agree as close as possible with

[2] The number of states belonging to a degenerate level can be inferred experimentally by placing the atom in an external magnetic field where the degeneracy with respect to the M quantum numbers is removed.

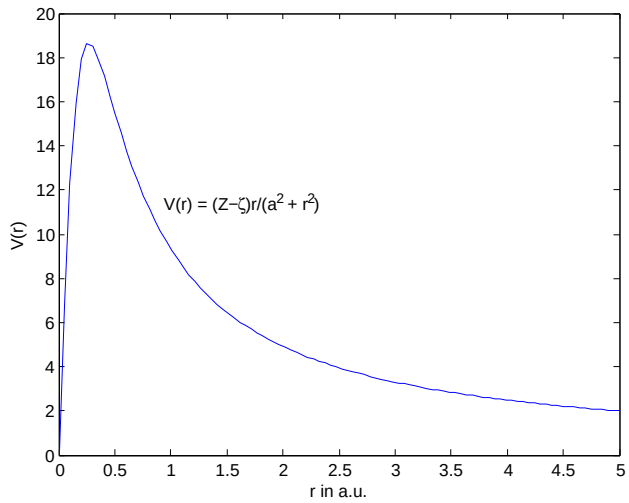


Figur 6: *Schematic diagram of the lowest configurations of Ne I. The levels of each configuration lie within the limited energy range shown by the shaded block.*

the experimental energies. Analyzing the potential we see that it indeed assumes the limiting values (29) and (30) just as it should. As an example we look at Na I with $Z = 11$ and $N = 11$ (that is $\zeta = 1$). In figure 6 the second part of the effective potential, the part related to the repulsive potentials of the remaining electrons, is displayed for $a = 0.2683$, a value that has shown to give a good agreement between computed energy levels and observed ones.

15 Sodium Energy Levels

As an example we will look at the energies in Na I. The ground configuration is $1s^2 2s^2 2p^6 3s$ and the excited configurations are given by $1s^2 2s^2 2p^6 3p$, $1s^2 2s^2 2p^6 3d$ etc. We modify the Python program in section 11 to handle the parametric potential above with $a = 0.2683$. Using the program we obtain the following energies



Figur 7: *Second part of the effective potential, the part related to the repulsive potentials of the remaining electrons.*

orbital	energy in a.u.
$1s$	-47.4653
$2s$	-1.6452
$2p$	-0.5532
$3s$	-0.1919
$3p$	-0.1072
$3d$	-0.0571
$4s$	-0.0720
$4p$	-0.0494
$4d$	-0.0321
$4f$	-0.0315
$5s$	-0.0377

From the energies we can compute the energies of the configurations. For the ground configuration $1s^2 2s^2 2p^6 3s$ we have

$$E(2p^6 3s) = 2E_{1s} + 2E_{2s} + 6E_{2p} + E_{3s} \tag{55}$$

The energy of the first excited configuration $1s^2 2s^2 2p^6 3p$ is given by

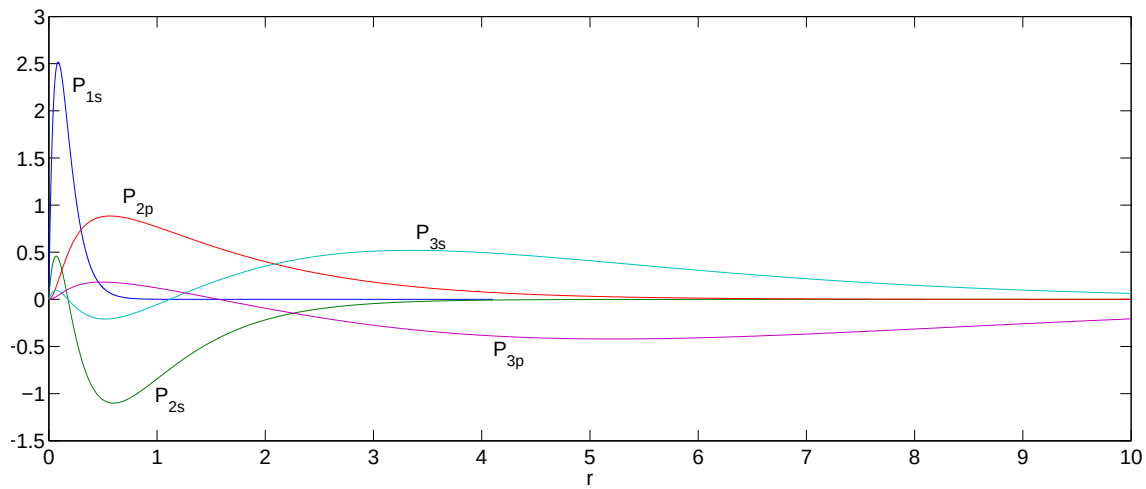
$$E(2p^6 3p) = 2E_{1s} + 2E_{2s} + 6E_{2p} + E_{3p} \tag{56}$$

Normally, the energies of a configuration is not observable. What can be observed is the energy difference between different configurations. Thus spectroscopic tables, such as the ones from NIST `physics.nist.gov/PhysRefData/ASD/levels_form.html`, give the energy differences between the excited states and the ground state. These differences should be compared with theory. Below we show an excerpt from the spectroscopic table for Na I. Energies are given in cm^{-1} where 1 a.u. corresponds to 219474.63 cm^{-1}.

Configuration	Term	J	Level
2p6.3s	2S	1/2	0.00000
2p6.3p	2P*	1/2	16956.17025
		3/2	16973.36619
2p6.4s	2S	1/2	25739.999
2p6.3d	2D	5/2	29172.837
		3/2	29172.887
2p6.4p	2P*	1/2	30266.99
		3/2	30272.58
2p6.5s	2S	1/2	33200.673
2p6.4d	2D	5/2	34548.729
		3/2	34548.764
2p6.4f	2F*	7/2	34586.92
		5/2	34586.92

Inserting the energies for the spin-orbitals and taking the energy differences, our simple model with a parametric potential gives the following values.

Configuration	Term	J	Level (NIST)	Calc.
2p6.3s	2S	1/2	0.00000	
2p6.3p	2P*	1/2	16956.17025	18589
		3/2	16973.36619	
2p6.4s	2S	1/2	25739.999	26315
2p6.3d	2D	5/2	29172.837	29585
		3/2	29172.887	
2p6.4p	2P*	1/2	30266.99	31275
		3/2	30272.58	
2p6.5s	2S	1/2	33200.673	33843



Figur 8: *Radial orbitals in Na I from calculations using a parametric potential.*

```
                |      |     |                |          |
2p6.4d          | 2D   | 5/2 |  34548.729     |   35072  |
                |      | 3/2 |  34548.764     |          |
                |      |     |                |          |
2p6.4f          | 2F*  | 7/2 |  34586.92      |   35204  |
                |      | 5/2 |  34586.92      |          |
-------------------------------------------------------|
```

We see that the consistency is quite good and that our model indeed gives the correct order of the configurations. We also note that, in this case, the energies depend quite strongly on the orbital quantum numbers so that, for example, $2p^6 4s$ lies below $2p^6 3d$. For highly charged ions energies are usually ordered by n.

16 Sodium Radial Orbitals

In figure 7 we show the radial functions (or radial orbitals) as predicted by our model. The radial orbital for $1s$ is strongly contracted and have a peak at around $r = 0.1$. Comparing the radial orbitals for $2s$ and $2p$ we see that the electron corresponding to $2s$ has some probability to be found close to the nucleus there it sees a more unscreened nuclear charge. As a result the energy of $2s$ will be lower than that of $2p$. The same arguments can be used to see why the energy of $3s$ is lower than $3p$ and $3d$.

17 References

The treatment is inspired by Walter R. Johnson, Atomic Structure Theory: Lectures on Atomic Physics, Springer Verlag, Berlin (ISBN 978-3-540-68010-9)

Part of the material can be found in C. Froese Fischer, T. Brage, and P. Jönsson,

Computational Atomic Structure, IoP, Bristol (ISBN 0-7503-0466-9)

The interested reader should consult R. Cowan, The Theory of Atomic Structure and Spectra, Univ. of California Press, Berkely (ISBN 0-520-03821-5)

18 Exercises and report

You should do the exercises below. Write a report, preferable in LaTeX using overleaf, where you:

1. give the commands used for solving the exercise
2. give the resulting output from Python. If you produce figures they should be included in the report. All figures should have a caption. Make sure that the font size of the figure axes are large enough to be read.
3. comment on the results as required in the exercise

If you are working in LaTeX it is a good idea to use the verbatim environment for the Python commands you use to solve the exercises, for example:

```
\begin{verbatim}
     import math
     import numpy as np
     import matplotlib.pyplot as plt

     r = np.linspace(0,30)
     p1s = 2*r*np.exp(-r)
     plt.plot(r,p1s)
     plt.show()
\end{verbatim}
```

Read the theory *very carefully* before you start with the exercises. If you are unfamiliar with Python check `https://www.python.org`. Save all your commands and the outputs in a Word-document or something similar.

1. Set $Z = 1$ and plot the hydrogenic orbitals $P_{1s}(r), \ldots, P_{3d}(r)$ in the same figure. Test different intervals for r until you are happy with the result. Use `gtext` to interactively put text in the plot.
2. Plot orbitals $P_{1s}(r)$ in the same figure for $Z = 1, 2, 3, 4$.
3. What is meant by the radial density function $D(r)$ and how do you interpret it? Plot the radial density functions for $3s, 3p, 3d$ and $Z = 1$ in the same plot. Carefully note how the $3s$ has a larger probability to be close to the nucleus. Compare Fig 3.6 in Cowan.
4. Download, install and import SymPy module in Python

5. Using SymPy functions to perform the integrations, verify that the spherical harmonics given on page 2 satisfy the following orthogonality relations

$$\int_{\varphi=0}^{2\pi}\int_{\theta=0}^{\pi} Y^*_{\ell m}(\theta,\varphi)Y_{\ell' m'}(\theta,\varphi)\sin\theta\, d\theta d\varphi = \delta_{\ell\ell'}\delta_{mm'}$$

6. Do the detailed mathematical derivations to verify the update formula (32).

7. Carefully study the program `radial.py` in section 10 so you know what is going on.

8. Set $Z = 1$ and plot the effective potential $V(r) = -Z/r + \frac{\ell(\ell+1)}{2r^2}$ for s, p, d, and f electrons (see figure 4). Plot all the potentials in the same plot and test different intervals for r until the plot looks nice.

9. Use the program `radial.py` with $Z = 1$ and generate energies and radial functions and for $1s, 2s, 3s, 4s, 6s, 9s$. Check that the numbers of nodes are correct. Also verify that the computed energies are in accordance with the energies from the analytical formula $E_n = -\frac{Z^2}{2n^2}$.

10. Derive the transformed radial equation (35). Hint: set $P(r) = \sqrt{r}\ \overline{P}(\rho)$ and express $P(r)$ and $P''(r)$ in eq. (7) in terms of $\overline{P}(\rho)$ and it's derivatives.

11. Study the program `radiallog.py` in detail.

12. Use the program `radiallog.py` with $Z = 1$ and generate energies and radial functions for $1s, 2s, 3s, 4s, 6s, 9s$. Compare the accuracy with what you get with `radial.py`. For comparison print the number of gridpoints used by `radiallog.py`.

13. Use the program `radiallog.py` with $Z = 1$ and generate energies and radial functions for $1s, 2s, 3s, 4s, 6s, 9s$. One simple python script to make multiple plots on one figure:

```
import numpy as np
import matplotlib.pyplot as plt
import radiallog

plt.figure()
result={}
for n in [1,2,3,4,6,9]:
    r,P=radiallog.radiallog(0,n,1,False,)
    plt.plot(r,np.sqrt(r)*P)

plt.legend(['1s','2s','3s','4s','6s','9s'])
plt.xlabel("r (a.u.)")
plt.ylabel("P")
plt.show()
```

You can use `plt.xlim` to get a suitable scale.

14. Use the program `radiallog.py` with $Z = 1$ and generate energies and radial functions and for $2p, 3p, 4p, 5p, 7p, 9p$. Check that the number of nodes are correct. Also verify the analytical formula $E_n = -\frac{Z}{2n^2}$ for the energy.

15. Look at the effective potential curve in exercise 9 and explain why the s orbitals have a steeper slope at the origin than do the p orbitals and why, as a consequence, they have higher probability densities close to the nucleus.

16. The radial expectation value

$$\langle r \rangle = \int_0^\infty r P_{n\ell}^2(r)\, dr \tag{57}$$

is a measure of the orbital radius. Compare, in a few cases, this value with the corresponding plot of the radial function to get some feeling for the meaning of the expectation value. Compare your computed value of $\langle r \rangle$ by the value you get by integrating the analytical expressions for the used orbitals. Use SymPy for the integration.

17. If you have time. Write a small program that uses Cowell-Numerov's method to integrate the solution further and further out to the practical infinity. Use $Z = 1$, $l = 0$, $n = 1$ and a starting energy value $E = -1/2$ (exact). Print the solution. You can use `ylim` to get a suitable scale.

18. Modify the program `radiallog.py` so that it uses the parametric potential $V_a(r) = -\frac{Z}{r} + \frac{(Z-\zeta)r}{a^2+r^2}$ instead of the Coulomb potential. *Don't forget to modify also the g function for Numerov.*

19. Perform calculations for some states Na I. Verify that the values given section 13 are correct. Plot some of the radial functions.

20. Perform calculations for Al III, i.e. $Z = 13$ and $N = 11$. Modify the value of a so that your energy differences fits those reported by NIST. Create tables similar to the ones in section 13. Plot the wave functions. Why do the configurations come in the "correct" order for this ion?

21. Perform calculations for Li I. Modify the value of a so that your energy differences fits those reported by NIST. Can you do some additional programing to automate this testing in some way? How could you do this? Create tables similar to the ones in section 13. Plot the wave functions.

Appendix: separation of variables

The wave function is $\phi(\mathbf{r},\sigma) = R(r)Y(\theta,\varphi)\chi_{m_s}(\sigma) = \frac{1}{r}P(r)Y(\theta,\varphi)\chi_{m_s}(\sigma)$. Inserting into equation (3) we get

$$\frac{1}{r^2}\frac{\partial}{\partial r}\left(r^2\frac{\partial R(r)}{\partial r}\right)Y(\theta,\varphi) + \frac{1}{r^2\sin\theta}\frac{\partial}{\partial\theta}\left(\sin\theta\frac{\partial Y(\theta,\varphi)}{\partial\theta}\right)R(r) + \frac{1}{r^2\sin^2\theta}\frac{\partial^2 Y(\theta,\varphi)}{\partial\varphi^2}R(r) + \frac{2m}{\hbar^2}\left(E - V(r)\right)R(r)Y(\theta,\varphi) = 0. \quad (58)$$

Now multiply with r^2 and divide by $R(r)Y(\theta,\varphi)$

$$\frac{\frac{\partial}{\partial r}\left(r^2\frac{\partial R(r)}{\partial r}\right)}{R(r)} + \frac{\frac{1}{\sin\theta}\frac{\partial}{\partial\theta}\left(\sin\theta\frac{\partial Y(\theta,\varphi)}{\partial\theta}\right)}{Y(\theta,\varphi)} + \frac{\frac{1}{\sin^2\theta}\frac{\partial^2 Y(\theta,\varphi)}{\partial\varphi^2}}{Y(\theta,\varphi)} + \frac{2m}{\hbar^2}\left(E - V(r)\right)r^2 = 0. \quad (59)$$

Rearrange so that we have all terms depending on r to the left and all terms depending on (θ,φ) to the right

$$\frac{\frac{\partial}{\partial r}\left(r^2\frac{\partial R(r)}{\partial r}\right)}{R(r)} + \frac{2m}{\hbar^2}\left(E - V(r)\right)r^2 = -\frac{\frac{1}{\sin\theta}\frac{\partial}{\partial\theta}\left(\sin\theta\frac{\partial Y(\theta,\varphi)}{\partial\theta}\right)}{Y(\theta,\varphi)} - \frac{\frac{1}{\sin^2\theta}\frac{\partial^2 Y(\theta,\varphi)}{\partial\varphi^2}}{Y(\theta,\varphi)}. \quad (60)$$

Since the LHS depends only on r and the RHS only on (θ,φ) both sides must be constant. We call the constant the separation constant and denote it by λ. Thus we have two equations

$$\frac{1}{\sin\theta}\frac{\partial}{\partial\theta}\left(\sin\theta\frac{\partial Y(\theta,\varphi)}{\partial\theta}\right) + \frac{1}{\sin^2\theta}\frac{\partial^2 Y(\theta,\varphi)}{\partial\varphi^2} + \lambda Y(\theta,\varphi) = 0 \quad (61)$$

and

$$\frac{\partial}{\partial r}\left(r^2\frac{\partial R(r)}{\partial r}\right) + \frac{2m}{\hbar^2}\left(E - V(r)\right)r^2 R(r) = \lambda R(r). \quad (62)$$

Now utilizing the the fact that $R(r) = \frac{P(r)}{r}$ it is not difficult to show that the latter equation yields

$$\frac{d^2P(r)}{dr^2} + \frac{2m}{\hbar^2}\left(E - V(r) - \frac{\lambda\hbar^2}{2mr^2}\right)P(r) = 0. \quad (63)$$

Appendix: SymPy

SymPy is a Python library for symbolic mathematics. It allows for derivatives, integrals, limits, and series expansions in SymPy. Have a look at: `https://www.sympy.org/en/index.html`. The `integrals` module in SymPy implements methods to calculate definite and indefinite integrals of expressions.

1. integrate(f, x) returns the indefinite integral $\int f(x)dx$
2. integrate(f, (x, a, b)) returns the definite integral $\int_a^b f(x)dx$

Example 1. To take the derivative of the function

$$f(x) = (2x^5 + 3)\sin(2x)$$

Start python and type:

```
>>> x=Symbol('x')
>>> diff((2*x**5+3)*sin(2*x),x)
10*x**4*sin(2*x) + 2*(2*x**5 + 3)*cos(2*x)
```

Example 2. To perform the integration

$$\int_0^{\pi} (2x^5 + 3)\sin(2x)\, dx$$

```
>>> from sympy import *
>>> x=Symbol('x')
>>> integrate(2*x**5*sin(2*x),(x,0,pi))
-pi**5 - 15*pi/2 + 5*pi**3
```

Example 3. To perform the double integration

$$\int_0^2 \left(\int_0^{\pi} \left((2y^5 + 3)\sin(2x) + y \right) dx \right) dy$$

```
>>> from sympy import *
>>> x,y=symbols('x y')
>>> f=(2*y**5+3)*sin(2*x)+y
>>> integrate(f,(x,0,pi),(y,0,2))
2*pi
```